

Durante il nostro primo incontro, abbiamo dedicato del tempo allo studio dei requisiti minimi del progetto, pianificando anche la successiva suddivisione dei compiti. Ecco una descrizione dettagliata delle soluzioni implementate:

1. Una Docker per il lato server:

Per rendere il progetto facilmente eseguibile su dispositivi con sistemi operativi diversi (come MacBook e Windows), abbiamo deciso di utilizzare Docker. Abbiamo creato un file docker-compose.yml suddiviso in due container principali:

- **App:** Utilizza un'immagine di PHP 8.1 con Apache, mappata sulla porta 80:80 per renderla accessibile localmente;
- **Database (db):** Utilizza MySQL 8 con il nome di container "dreamcity-db", eseguendo lo script init.sql per la creazione automatica del database alla build.

2. Gestione degli utenti: login e loro informazioni (non sensibili):

Abbiamo implementato un sistema di login e registrazione per gli utenti, memorizzando i loro dati (username, password, ecc.) in una tabella del database. Per la sicurezza, abbiamo usato la funzione password_hash di PHP con l'algoritmo Argon2i per criptare le password, garantendo una maggiore protezione.

3. Tabella dei record:

Quando un utente posiziona una struttura, la felicità e il numero di abitanti possono aumentare. Nella schermata di selezione delle mappe, queste vengono ordinate in base a questo parametro, permettendo di mantenere una classifica aggiornata.

4. Invito ad amici per partecipare ad una partita:

Abbiamo integrato il sistema di inviti all'interno della casella messaggi. Gli utenti possono inviare messaggi di invito a una partita, che, se accettati, reindirizzeranno il destinatario direttamente alla mappa associata all'invito.

5. Una collezione di 10 mappe di 10000x10000 mattonelle tra cui scegliere:

Dopo alcune prove, abbiamo deciso di gestire le mappe tramite una grande div HTML di 10000x10000 pixel, con all'interno altre piccole div per rappresentare le strutture piazzate dagli utenti. Questo approccio ha ottimizzato la gestione della memoria e migliorato la resa visiva rispetto alla scelta iniziale di utilizzare Canvas di HTML5.

6. I giocatori possono giocare in modo asincrono. La partita, quindi, rimane attiva anche se non ci sono giocatori che stanno giocando. Viene distrutta solo con il consenso di tutti i giocatori o dopo un determinato periodo di inattività.:

Le partite rimangono attive anche in assenza di giocatori. Gli edifici vengono caricati automaticamente quando un utente accede alla mappa. Una partita può essere distrutta solo con il consenso di tutti i giocatori o dopo sette giorni di inattività.

7. È possibile cambiare un giocatore in corso d'opera.:

Abbiamo inserito un pulsante di logout disponibile sia nella schermata di selezione delle mappe che in quella di gioco, permettendo agli utenti di uscire dal loro profilo e accedere con un altro account.

8. **È possibile scambiarsi gli assessorati:**

Lo scambio dei ruoli tra giocatori è gestito tramite messaggi. Un utente può inviare una richiesta di scambio, e una volta accettata, il ruolo verrà aggiornato automaticamente, con la possibilità di accedere a nuove strutture in base al nuovo ruolo.

9. **Interfaccia per la creazione manuale di nuove mappe da caricare sul server e mettere a disposizione degli altri giocatori:**

Abbiamo creato un'interfaccia per la creazione di nuove mappe. Nella schermata di selezione delle mappe, gli utenti possono cliccare su "crea nuova mappa", compilare un modulo con nome, descrizione e immagine di copertina, e renderla disponibile agli altri giocatori una volta creata.

Questa suddivisione ci ha permesso di organizzare il lavoro in modo efficiente e coprire tutti i requisiti minimi richiesti dal progetto.